

med tech plan V1

Table of Contents

Med-Tech Plan — V1

Purpose of this file. This is the *master plan* (the "north star") for a Med-Tech education program that turns a healthcare student — physician, dentist, pharmacist — with basic Python into an **autonomous medical-AI developer**. It is written to be **retrieved and iterated on by AI models** across sessions, so the AI does not start from scratch each time. This document defines the philosophy, structure, topics, labs, knowledge, software specifications, and decision points; the detailed teaching content is built out from it, **chapter by chapter**, into the multi-volume book described in §2.

What this is becoming. This plan is being developed into a **multi-volume book and a formal university course in Med-Tech** — intended for adoption as a real curriculum in medical, dental, and pharmacy schools. Each part of the program becomes a **volume**; each volume is divided into numbered **chapters**; each chapter carries **formal curriculum definitions** (learning outcomes, a topic-by-topic content list, labs, and **theory + lab contact hours**). We will **expand this document chapter by chapter**, filling in the real list of topics that theory and labs pass through, with total hours for each.

- **Status:** V1 — living document, expected to iterate. Evolving from *plan* → *multi-volume textbook & university course*.
- **Companion (future):** docs/med-tech-curriculum-V1.md (detailed, teachable curriculum / instructor guide).
- **Change context:** ai-context/change-log.md (in-repo, per-iteration record of what changed and why — read it first, and append to it after each change).

- **Program length:** 12–15 months, three phases (plus a foundational Phase 0) and two strategic chapters, organized as **5 volumes / 27 chapters** (see §2.3).
 - **Owner intent:** capture reasoning and structure now; expand chapter by chapter into the formal book/curriculum.
-

1. Vision & Guiding Philosophy

1.1 The core thesis

It is **easier to teach a clinician the fundamentals of programming logic and AI development than to teach a seasoned AI/software engineer the depth of clinical and surgical medicine**. A medical or dental professional carries clinical, diagnostic, and procedural knowledge that an informatics or software professional does not have and cannot easily acquire. When that clinical depth is combined with enough AI fluency, the resulting output is **more accurate, safer, and better aligned with real clinical needs**.

1.2 What we are (and are not) building

- **We are NOT training the future doctor to become a software engineer.**
- **We ARE giving the clinician enough skill to:**
 - Fully understand the **terminology** of the AI world.
 - **Understand concepts deeply enough to measure and justify decisions** throughout the development process.
 - Become **independent enough** to create consistent, market-ready solutions **with a minimum need for a software developer**.
 - Contribute the clinical/health domain expertise that pure informatics professionals lack.

1.3 Realistic team assumption

No single person builds a production medical-AI LLM solution alone. But **a domain expert from medicine with very deep clinical knowledge — who is also AI-fluent**

— **produces a more accurate and trustworthy output** and can lead, specify, and validate the work with far less dependence on external engineers.

1.4 Target audience

Students and professionals in the health sciences: physicians, dentists, pharmacists, and analogous clinical fields, who already have **basic Python** knowledge and clinical/surgical training (or are on the path to it).

1.5 End-state outcome

By the end of the program the student can **create medical LLMs autonomously — both training and inference** — including fine-tuning, RAG grounding, evaluation, safe deployment, and awareness of the regulatory/ethical landscape.

2. Program Structure & Academic Format

2.1 A multi-volume book and a university course

This program is authored as a **multi-volume textbook** and a **formal university course** for Med-Tech, suitable for adoption in medical, dental, and pharmacy schools. The academic architecture is:

- **Volume** — a major part of the program (maps to a phase or strategic chapter). 5 volumes total.
- **Chapter** — a numbered teaching unit inside a volume, with a **formal curriculum definition** (see §2.5). 27 chapters total.
- **Topic** — an individual item of theory content inside a chapter; each topic is taught through **theory** and reinforced by **labs**.
- **Lab** — a hands-on, assessable exercise attached to a chapter.

The detailed phase write-ups later in this document (§3–§7) are the **source material** for these chapters; we expand each chapter from them, topic by topic, adding the real content list and hour counts.

2.2 Phase overview (the three-phase spine)

Phase	Name	Months	Maps to
0	The Foundational Bridge	1–3	Volume I
1	Ascending to the LLM Era	4–6	Volume II
2	Autonomous Medical AI Developer	7–15+	Volume III
★	Strategic: From Books to LLM	bridges 1→2	Volume IV
★	Strategic: AI for Medical Robots	elective/frontier	Volume V

2.3 Volume & chapter map (with contact-hour budget)

Hours are provisional planning estimates (theory = lecture/seminar contact hours; lab = supervised hands-on hours). They are the budget we refine as each chapter is expanded (§2.5). T = theory h, L = lab h.

Volume I — Foundations & Clinical Data Engineering (*Phase 0*)

Ch	Title	T	L	Total
0	Python & GitHub Fundamentals (onboarding prerequisite; +40 h)	10	20	30

Ch	Title	T	L	Total
	<i>homework)</i>			
1	Python for Data Science (NumPy, Pandas, Matplotlib)	12	18	30
2	Foundations of Clinical Data (EHR; ICD/CPT/SN OMED/LOINC; HIPAA/GDPR)	10	6	16
3	Healthcare Interoperability (HL7 v2, CDA/C-CDA, FHIR, SMART-on-FHIR)	12	16	28
4	Core Machine Learning for Medicine (scikit-learn)	14	16	30
5	Medical NLP Basics	10	10	20

Ch	Title	T	L	Total
	(regex → embeddings)			
	Volume I subtotal (Ch 1–5)	58	66	124

Chapter 0 is an **onboarding prerequisite** (Python + Git/GitHub from scratch). It uses a distinct **Theory : Labs : Homework = 1 : 2 : 4** model (10 h + 20 h + **40 h homework = 70 h**) and is the first chapter to introduce the **Homework** component. Its content lives in the per-chapter tree (docs/Volume I/Chapter 0/). Because it is pre-course prep with a different model, its hours are **not folded into** the 124 h Volume I core subtotal or the program grand totals — track it separately.

Volume II — The LLM Era (*Phase 1*)

Ch	Title	T	L	Total
6	Transformer Architecture & Self-Attention	12	6	18
7	The Hugging Face Ecosystem	8	12	20
8	Prompt Engineering for Medicine	8	10	18
9	Medical Domain	10	6	16

Ch	Title	T	L	Total
	Adaptation (Meditron, Clinical- Camel)			
10	Benchmarki ng & Hallucinatio n (MedQA, PubMedQA)	8	8	16
11	First Fine- Tunes (PubMedBE RT, QLoRA, local MedGemma /Ollama)	10	20	30
	Volume II subtotal	56	62	118

Volume III — Autonomous Medical AI Systems *(Phase 2)*

Ch	Title	T	L	Total
12	Retrieval- Augmented Generation (RAG)	12	18	30
13	Vector Databases & Knowledge	8	12	20

Ch	Title	T	L	Total
	Storage			
14	Training Pipeline: Pre-training → Fine-tuning → Alignment (DPO)	16	20	36
15	Evaluation, Bias, Fairness & Clinical Safety	12	10	22
16	Deployment, FHIR Integration & Compliance	10	16	26
17	Capstone Project	6	60	66
	Volume III subtotal	64	136	200

Volume IV — From Books to LLM (*Strategic §6*)

Ch	Title	T	L	Total
18	Ingestion, OCR & Document Understanding	8	14	22

Ch	Title	T	L	Total
	ng			
19	Figures & Multimodal Understanding	8	14	22
20	Structuring, Provenance & Building the Corpus	6	12	18
21	RAG-vs- Fine-tune Paths & Synthetic Data	8	16	24
22	State of the Art & Evaluation (text/image/video)	8	6	14
	Volume IV subtotal	38	62	100

Volume V — AI for Medical Robots (*Strategic §7 — elective/frontier*)

Ch	Title	T	L	Total
23	Robotics Fundamentals & Simulation (ROS 2,	10	16	26

Ch	Title	T	L	Total
	Isaac Sim)			
24	Perception for Clinical/Surgical Scenes	8	14	22
25	Learning for Control (Imitation/RL, sim-to-real)	10	14	24
26	LLMs/VLA as Planners & Human-in-the-Loop Safety	8	12	20
27	Regulation, Ethics & Liability	8	4	12
	Volume V subtotal	44	60	104

Program totals (provisional): Core (Volumes I–IV): **T 216 h + L 326 h = 542 contact hours**. With elective Volume V: **646 contact hours**.

2.4 Hour & credit conventions

- **Theory hours (T):** instructor-led lecture/seminar time on the topic list.
- **Lab hours (L):** supervised, assessable hands-on work (the labs listed per chapter).
- **Total contact = T + L.** Independent self-study is additional (guideline: $\approx$ equal to contact hours).

- **Credit mapping (indicative):** using **1 ECTS $\approx$ 25–30 total student-effort hours** (contact + self-study), the core (~542 contact h + comparable self-study) is roughly **40–52 ECTS**; the elective adds ****~8 ECTS****. Institutions should map to their own credit system.
- All hour figures in §2.3 are **provisional** until the corresponding chapter is expanded per §2.5.

2.5 Formal chapter definition (template to expand each chapter)

Every chapter is filled out using this schema. Expanding the book = completing one of these per chapter, with the **real topic list** and **per-topic hours**.

```
Chapter  N  –  <Title>                                [Volume  X]
Prerequisites:                <chapters/skills      required      first>
Learning outcomes: <what the student can DO after this chapter
(measurable                                     verbs)>
Topics (theory):    <ordered list of topics; hours per topic; sums to
chapter                                                    T>
Labs (hands-on):    <ordered labs; hours per lab; sums to chapter L>
Datasets/tools:      <datasets,  libraries,  models  used>
Assessment:          <quiz / lab rubric / project deliverable>
Key decisions:       <engineering/clinical trade-offs taught here>
References:          <pointers      into      §13>
Hours:               Theory <T> + Lab <L> = <Total>
```

Strategic chapters map to Volumes IV (§6) and V (§7). **Guiding principle throughout:** the clinician's background is the *superpower* — every topic and lab is anchored to a real clinical problem.

2.6 Worked example — Chapter 3 fully defined

A reference implementation of the §2.5 template, showing the level of detail each expanded chapter should reach (topic-by-topic hours that sum to the chapter budget). The remaining 26 chapters will be filled out to this standard.

Chapter 3 – Healthcare Interoperability (HL7 v2, CDA/C-CDA, FHIR, SMART-on-FHIR) [Volume I]

Prerequisites: Ch 1 (Python/Pandas), Ch 2 (clinical data & code sets)

Learning outcomes – the student can:

- Explain how clinical data is exchanged (not just stored) and why it is the backbone of medical AI.
- Read and parse an HL7 v2 message and a C-CDA document into structured data.
- Model clinical data as FHIR resources and query a FHIR server via its REST API from Python.
- Bind data to standard terminologies (ICD/SNOMED/LOINC) and describe a SMART-on-FHIR integration.

Topics (theory) – 12 h:

3.1 Why interoperability matters; the data backbone of medical-AI products (1 h)

3.2 HL7 v2.x messaging: segments, trigger events (ADT/ORM/ORU), pipe-delimited format (2 h)

3.3 HL7 CDA & C-CDA: XML clinical documents, templates, sections (2 h)

3.4 FHIR fundamentals: resources, references, bundles, data types (3 h)

3.5 FHIR REST API: search, read, CRUD, versioning, pagination (2 h)

3.6 Terminology binding in FHIR (ICD/SNOMED/LOINC); profiles & conformance (1 h)

3.7 SMART-on-FHIR & app integration; OAuth scopes, security basics (1 h)

Labs (hands-on) – 16 h:

Lab 3a Parse an HL7 v2 message with `hl7apy` into structured fields (3 h)

Lab 3b Read a C-CDA document; extract problems / medications / allergies (3 h)

Lab 3c Stand up a HAPI FHIR server and load Synthea synthetic patients (3 h)

Lab 3d Query the FHIR REST API from Python; Patient/Observation/Condition → Pandas (4 h)

Lab 3e Build a patient-summary extractor over FHIR (mini SMART-on-FHIR read flow) (3 h)

Datasets/tools: Synthea synthetic patients; HAPI FHIR (public sandbox or local); Python `hl7apy`, `fhir.resources`/`fhirclient`, `requests`, Pandas; sample C-CDA XML.

Assessment: Quiz on standards (20%) + lab rubric on 3a–3e (50%) + a "clinical data → tidy DataFrame" deliverable from a FHIR bundle (30%).

Key decisions: FHIR vs. HL7 v2 for a new integration; document (CDA) vs. resource (FHIR) models; how much to normalize/terminology-map before feeding an AI pipeline.

References: §13 → Interoperability standards.

Hours: Theory 12 + Lab 16 = 28

3. Phase 0 — The Foundational Bridge (Months 1–3)

Goal: combine essential programming skills with a deep understanding of the clinical problems to be solved. Build the classical-ML foundation before touching LLMs.

3.1 Topics to master

1. **Python for Data Science (review & deepen).** Move beyond basics. Core AI libraries: **NumPy**, **Pandas**, **Matplotlib**, **Scikit-learn**.
2. **Foundations of Clinical Data.** Structure of **Electronic Health Records (EHRs)**; medical code sets (**ICD**, **CPT**, **SNOMED**, plus **LOINC** for labs); critical importance of **data privacy (HIPAA / GDPR)**.
3. **Healthcare interoperability standards.** How clinical data is *exchanged*, not just stored — the backbone of any real-world medical-AI product:
 - **HL7 v2.x** — the legacy messaging standard (pipe-delimited segments: ADT, ORM, ORU) still dominant in hospitals.
 - **HL7 CDA (Clinical Document Architecture)** — XML-based clinical documents (e.g., C-CDA continuity-of-care documents).
 - **HL7 FHIR (Fast Healthcare Interoperability Resources)** — the modern **REST/JSON** standard; understand **Resources** (Patient, Observation, Condition, Encounter, MedicationRequest), references, bundles, and the FHIR REST API. This is the primary target for new development.
4. **Core Machine Learning for Medicine.** Fundamentals of **supervised and unsupervised learning**; applications in **clinical risk prediction, patient stratification, diagnostics**.
5. **Medical NLP basics.** Traditional NLP: **regular expressions** for clinical text processing through to **word embeddings**.

3.2 Hands-on labs

- **Lab 1 — EHR Data Wrangling.** Use Pandas to load, clean, and analyze a **synthetic EHR dataset**. Practice **feature engineering** and handle common clinical-data challenges.
- **Lab 2 — Simple Clinical Classifier.** Train a model (**Logistic Regression**, **Random Forest**) to predict a clinical outcome (e.g., **hospital readmission**) using structured EHR data, with **Scikit-learn**.

- **Lab 2b — FHIR & Interoperability Hands-on.** Stand up a test **FHIR server** (e.g., **HAPI FHIR** public sandbox or **Synthea**-generated synthetic patients). Query the **FHIR REST API** with Python (`requests / fhir.resources / fhirclient`), parse **Patient / Observation / Condition** resources into a Pandas DataFrame, and parse a sample **HL7 v2 message** (with `hl7apy`) and a **C-CDA** document. Goal: turn interoperability standards into model-ready data.

3.3 Key decisions

- **Software stack:** settle on a Python environment (**Anaconda** is great for beginners). Install the core data-science stack: `pandas, numpy, matplotlib, scikit-learn, jupyter`.
 - **Hardware:** a **standard laptop or desktop** is more than sufficient. Focus on concepts and workflows, not compute.
-

4. Phase 1 — Ascending to the LLM Era (Months 4–6)

Goal: make the leap from classical ML to Large Language Models.

4.1 Topics to master

1. **Transformer architecture.** Understand **self-attention** — the foundation of all modern LLMs.
2. **The Hugging Face ecosystem.** Proficiency with the **Transformers** and **Datasets** libraries; the portal to thousands of pre-trained models.
3. **Prompt engineering for medicine.** Techniques for accurate, useful responses: **chain-of-thought**, **few-shot prompting**, and how to **assess model reliability**.
4. **Medical domain adaptation.** How a general LLM is adapted to medicine; study models like **Meditron** and **Clinical-Camel** to see how **continued pre-training on biomedical text** is done.
5. **Benchmarking medical LLMs.** Evaluation on clinical-knowledge recall (**MedQA**) and context-grounded QA; the critical issue of **model hallucination**.

4.2 Hands-on labs

- **Lab 3 — The Local Medical Assistant (pivotal).** Install **Ollama** and run **MedGemma** (Google's open medical model) **entirely on the local machine** — an immediate, tangible understanding of local AI deployment.
- **Lab 4 — The Hugging Face Playground.** Load a pre-trained biomedical BERT (e.g., **PubMedBERT**) and **fine-tune it** on a specific task, such as **classifying the intent of a clinical question**.
- **Lab 5 — Fine-Tuning Your First Medical LLM.** Using transformers and a smaller model (**GPT-2** or a small **Llama** variant), apply **QLoRA** to fine-tune on a **medical question-answering dataset** (e.g., Hugging Face `medical_qa`). Demonstrates adapting a model with limited hardware.

4.3 Key decisions

- **Hardware.** A workstation with a modern GPU (e.g., an **RTX-5080-class, ~16 GB VRAM**) is excellent for this phase — well-suited to fine-tuning smaller models and running MedGemma. Larger models will hit its limits, foreshadowing the Phase 2 hardware decision.
-

5. Phase 2 — Autonomous Medical AI Developer (Months 7–15+)

Goal: move from running/fine-tuning models to building **robust, production-ready systems** — the most critical phase.

5.1 Topics to master

1. **Retrieval-Augmented Generation (RAG).** Ground an LLM in your own medical knowledge base. Build a RAG pipeline that retrieves from **clinical guidelines, textbooks, or medical literature** to produce **evidence-based responses**, minimizing hallucination.
2. **Vector databases & knowledge storage.** Store medical knowledge as **vector embeddings** in specialized databases (**Chroma, Weaviate, Pinecone**).

Understand *where knowledge lives in hardware* — often **RAM or NVMe** for fast retrieval.

3. **Model training — from fine-tuning to pre-training.** The full three-phase pipeline:
 - **General pre-training** (vast text exposure),
 - **Domain-specific fine-tuning** (medical corpora, e.g., **MedFineWeb**),
 - **Preference alignment** (e.g., **DPO**) for helpful, safe outputs.
4. **Model evaluation & safety.** Rigorous evaluation: **accuracy, bias, fairness, clinical safety**. Understand why benchmark performance (e.g., **MedQA**) **may not translate to a clinical setting**.
5. **Deployment & compliance.** Deploy locally with tools like **Ollama** and **Open WebUI**; monitor performance; navigate the **regulatory and ethical landscape** of AI in healthcare. **Integrate via FHIR**: expose/consume a **FHIR API** so the AI system reads patient context and writes results back into standards-based EHR workflows (e.g., SMART-on-FHIR-style integration), the key to a market-ready, interoperable product.

5.2 The capstone project

The final project must involve **training and building a deployable AI system for a real-world clinical problem**.

Example — "Clinical Trial Matching Assistant":

1. **Gather knowledge.** Use a RAG pipeline to ingest and index a database of clinical-trial descriptions.
2. **Train / adapt.** Fine-tune an LLM (e.g., a small Llama) on patient summaries and relevant trials.
3. **Inference server.** Deploy locally on the chosen hardware stack. Input: a patient's **synthetic** medical history. Output: a **ranked list of relevant clinical trials, citing evidence** from the knowledge base.

5.3 Key decisions — the final hardware decision

- **Mac Studio (M4 Max class). 64 GB+ unified memory** is a game-changer for running massive models and large context windows (e.g., full patient histories). Excels at **inference** and can host large **RAG vector databases in memory** for fast retrieval. A powerful, quiet, efficient all-in-one for running and serving models — well-suited to **autonomous development and deployment**.
- **Windows PC (RTX-class, upgrade path to RTX 6000 Ada, 48 GB VRAM)**. A powerhouse for **training and fine-tuning**. The **CUDA ecosystem** remains the industry standard for training and many advanced frameworks. Better if the plan involves **very heavy, compute-intensive training/fine-tuning** on a single workstation.

Decision heuristic: optimize the Mac path for **inference + serving + large-context RAG**; optimize the CUDA path for **heavy training/fine-tuning**.

6. Strategic Chapter — From Books to LLM

Why this is strategic. This is one of the highest-leverage skills in the whole program: taking authoritative medical knowledge that lives in **books, atlases, and journals — text and images** — and converting it into an **accurate, evidence-grounded medical LLM**. The clinician's judgment about *what is authoritative* and *what "accurate" means clinically* is exactly the edge a pure engineer lacks. Positioned as a bridge from Phase 1 into Phase 2, deepened alongside RAG and training.

6.1 The end-to-end pipeline (source → model)

1. **Ingestion & digitization.** Parse PDFs and scanned books: layout analysis, reading order, columns, headers/footers, tables, footnotes, references.
2. **OCR for medical text.** Handle domain-specific vocabulary, drug names, dosages, formulas, and units; scanned/low-quality pages; multi-language

sources. (Tools: Tesseract, and modern document/vision models; layout tools like unstructured, PyMuPDF, Marker, Nougat.)

3. **Figure & image extraction and understanding.** Detect and extract medical illustrations, radiographs, histopathology, ECGs, diagrams, and charts; generate/curate **captions**; link each figure to its surrounding text so the image and its explanation stay connected.
4. **Structuring & provenance.** Chunk content semantically; attach **metadata and citations** (book, edition, page, figure number) so every fact is **traceable** — essential for evidence-based, non-hallucinated output.
5. **Two downstream paths (a core decision):**
 - **RAG knowledge base** — embed chunks (text + image captions/multimodal embeddings) into a vector DB for *grounded, cited retrieval*. Lower risk, updatable, keeps provenance. **Default first choice for factual accuracy.**
 - **Training corpus** — use the extracted material for **continued pre-training / fine-tuning / instruction tuning**, including **synthetic Q&A generation** from the book content and **image-text pairs** for multimodal training. Higher effort/cost; changes model weights.
6. **Evaluation loop.** Test factual fidelity against the source, citation correctness, and clinical validity — not just fluency.

6.2 Key challenges (discuss explicitly with students)

- **Accuracy & hallucination.** The bar in medicine is high; ungrounded generation is dangerous. Favor RAG + citations; measure faithfulness.
- **OCR/parsing errors** on tables, equations, and specialized terminology corrupt downstream data.
- **Image understanding is hard.** Radiology/pathology/figure interpretation is far less mature than text; captions may be incomplete or wrong.
- **Copyright & licensing (critical, non-technical).** Most medical textbooks are **copyrighted**. Ingesting them for training or redistribution has real legal limits.

Use **licensed, open-access** (e.g., **open-access journals, public guidelines**), or **owned** content; get permissions; keep this front-and-center. (*This is a decision the clinician-led team must own, not defer.*)

- **Provenance & updatability.** Medicine changes; knowledge must be versioned and re-citable.
- **Cost & compute** of multimodal training vs. the cheaper, safer RAG route.

6.3 State of the art today (calibrated, ~2026)

- **Text medical LLMs:** strong and improving — e.g., **Med-PaLM 2, Meditron, MedGemma**, and frontier general models with strong medical performance. They pass medical-exam-style benchmarks well, but **benchmark ≠ clinical safety**.
- **Multimodal (image + text):** rapidly advancing but **less reliable** than text — e.g., **Med-PaLM M, LLaVA-Med**, multimodal **MedGemma**, radiology-focused models (chest-X-ray report generation), and **pathology foundation models**. Useful for assistance/drafting, **not autonomous diagnosis**.
- **Video:** **earliest/least mature** — surgical-workflow and endoscopy/echocardiography understanding are active research, not production-grade.
- **Honest framing for students:** the field is real and fast-moving, but **image/video medical understanding lags text**, and **all of it requires human oversight, grounding, and rigorous evaluation** before clinical use.

6.4 Labs

- **Lab 6 — Book → RAG.** Take a licensed/open medical text, run the full ingest→OCR→chunk→embed→retrieve pipeline, and answer questions **with page-level citations**.
- **Lab 7 — Figures that talk.** Extract figures, generate captions with a multimodal model, and make images retrievable alongside their text.
- **Lab 8 — Synthetic instruction data.** Auto-generate Q&A pairs from the source and use them to fine-tune a small model (compare against the RAG baseline for accuracy and hallucination).

6.5 Key decisions

- **RAG vs. fine-tune vs. both** (start RAG for accuracy; fine-tune for style/latency/offline).
 - **Licensing/ownership of every source** before ingestion.
 - **Multimodal model choice** and how much image understanding the product actually needs.
-

7. Strategic Chapter — AI for Medical Robots

Scope. Where modern AI (perception, planning, and increasingly LLM/VLA models) meets **embodied medical systems**: surgical robotics, assistive/rehabilitation robots, autonomous lab and diagnostic automation, and telepresence. This is a **frontier/advanced elective** that builds on the core program; it is deliberately honest that clinical autonomy here is still limited.

7.1 Topics to master

1. **Robotics fundamentals.** Kinematics, control loops, real-time constraints, and the software backbone (**ROS 2**), plus simulation (**Gazebo, NVIDIA Isaac Sim**).
2. **Perception.** Computer vision for surgical/clinical scenes: instrument tracking, tissue/organ segmentation, depth, pose estimation.
3. **Learning for control.** Imitation learning and reinforcement learning; **sim-to-real** transfer; safety constraints.
4. **LLMs & foundation models in robotics. Vision-Language-Action (VLA)** models and LLMs as **high-level planners/reasoners** that turn instructions and scene understanding into robot actions — an emerging paradigm, largely research-stage in medicine.
5. **Human-in-the-loop & safety.** Teleoperation vs. supervised autonomy vs. full autonomy; haptics; fail-safes; verification.
6. **Regulation, ethics & liability.** Far higher stakes than software-only AI — physical risk, device regulation (FDA/CE), accountability, and rigorous validation.

7.2 State of the art today (calibrated, ~2026)

- Most clinical surgical robotics (e.g., da-Vinci-class systems) is **teleoperated**, not autonomous — the AI assists (vision, guidance, ergonomics) rather than acts alone.
- **Task-level autonomy** exists mainly in **research** (e.g., autonomous suturing/soft-tissue demos like STAR) under tight constraints.
- **VLA / robot foundation models** are advancing quickly in general robotics but are **early and unproven** in safety-critical medicine.
- **Honest framing:** promise is large, but clinical deployment is gated by **safety, real-time reliability, data scarcity, and regulation**. Students should design for **human oversight by default**.

7.3 Challenges

- Safety-critical, real-time, physically irreversible actions.
- Data scarcity and the difficulty/cost of high-fidelity simulation and sim-to-real.
- Generalization across patients/anatomy; edge cases.
- Regulatory approval and liability for autonomous action.

7.4 Labs (simulation-first)

- **Lab 9 — ROS 2 + simulation basics.** Bring up a simulated arm in Isaac Sim/Gazebo; command basic motions.
- **Lab 10 — Perception task.** Segment/track instruments or targets in a simulated surgical scene.
- **Lab 11 — LLM-as-planner (sim).** Use an LLM/VLA to translate a natural-language task into a safe, verified action sequence in simulation, **with a human-approval gate**.

7.5 Key decisions

- **Simulation vs. hardware** (start in simulation; hardware only with proper safety/regulatory footing).

- **Autonomy level** appropriate to the clinical task and its risk.
 - **Compute** (GPU workstation for simulation/training; the Phase 2 hardware guidance applies).
-

8. Cross-Cutting Knowledge (spans all phases)

- **Clinical data literacy:** EHR structure, ICD/CPT/SNOMED/LOINC code sets.
 - **Interoperability standards:** **HL7 v2.x**, **HL7 CDA / C-CDA**, and **FHIR** (REST/JSON resources) — the data backbone for ingesting real clinical data into AI pipelines and integrating solutions with hospital systems.
 - **Privacy, security, compliance:** HIPAA, GDPR; synthetic-data practice before any real data.
 - **Evaluation & safety mindset:** hallucination, bias, fairness, clinical validity vs. benchmark scores.
 - **Reproducibility & tooling:** environment management, notebooks, version control.
 - **Ethics & regulation:** the healthcare-AI regulatory landscape and responsible deployment.
-

9. Software Specifications (consolidated)

Layer	Tools / Frameworks
Environment	Anaconda / Python, Jupyter
Data science	NumPy, Pandas, Matplotlib, Scikit-learn
Interoperability	FHIR (HAPI FHIR server, fhir.resources, fhirclient), HL7 v2 (hl7apy), CDA / C-CDA (XML), Synthea synthetic patients

Layer	Tools / Frameworks
NLP / LLM	Hugging Face Transformers & Datasets , biomedical BERT (PubMedBERT)
Fine-tuning	QLoRA , transformers; small models (GPT-2, small Llama)
Local inference	Ollama , MedGemma , Open WebUI
RAG / vectors	Chroma , Weaviate , Pinecone ; embeddings on RAM/NVMe
Alignment / training	Domain fine-tuning (MedFineWeb), preference alignment (DPO)
Reference models	Meditron , Clinical-Camel (domain adaptation exemplars)

10. Hardware Specifications (by phase)

Phase	Recommended hardware	Rationale
0	Standard laptop/desktop	Concepts & workflows; no heavy compute
1	GPU workstation (~16 GB VRAM, RTX-5080 class)	Fine-tune small models; run MedGemma locally
2 (inference/serving)	Mac Studio (M4 Max, 64 GB+ unified memory)	Large models, long context, in-memory RAG, quiet all-in-one

Phase	Recommended hardware	Rationale
2 (heavy training)	Windows/CUDA workstation (RTX 6000 Ada, 48 GB VRAM)	Industry-standard CUDA for compute-intensive training

11. Milestone Checklist (path forward)

1. **Master the basics** — solidify Python and foundational ML.
2. **Run MedGemma locally** — complete the tutorial; understand local deployment.
3. **Fine-tune a model** — successfully fine-tune a small model on a medical dataset using QLoRA.
4. **Build a RAG pipeline** — answer questions from a custom medical knowledge base.
5. **Books → LLM** — convert a licensed/open medical text (with figures) into a grounded, cited knowledge base or fine-tuned model (§6).
6. **Complete the capstone** — build and present an end-to-end medical-AI project.
7. **(Elective) Medical robotics in simulation** — complete the sim-first robotics labs (§7).

Outcome: the student is not only "updated and fluent" in AI, but has the practical skills to **create, train, and deploy medical LLMs autonomously**.

12. How AI Assistants Should Use This File

- **Read `ai-context/change-log.md` first**, alongside this plan, to load the context of prior iterations *without* re-reading the whole git history (token-efficient). **After making any change to project content, append a concise entry to that log**

(newest first: date, one-line summary, the *why*, files touched). Git holds the exact diffs; the log holds the reasoning.

- Treat this as the **authoritative source of intent** for the Med-Tech program. Do **not** re-derive the philosophy or structure from scratch each session — build on it.
- When asked to produce or refine the **curriculum**, translate these phases/topics/labs/decisions into med-tech-curriculum-V1.md (week-by-week, learning objectives, assessments, datasets, and step-by-step lab instructions).
- Preserve the **core thesis** (clinician-first) and the **explicit non-goal** (not turning doctors into software engineers) in all downstream artifacts.
- When information conflicts or is uncertain, surface it to the owner rather than silently overriding this plan.
- Version new major revisions as -V2, -V3, ... keeping prior versions for traceability.

- **Default export behavior:** whenever a **new version** of the plan or the curriculum is finalized, **generate matching .docx and .pdf** artifacts at the end so the repo always ships Word + PDF alongside the Markdown. Use the repeatable script:

```
scripts/export-docs.sh                                # exports current plan + curriculum
scripts/export-docs.sh docs/<file>.md # exports a specific file
```

(Pipeline: pandoc md→docx, then LibreOffice soffice docx→pdf. Keep the exported .docx/.pdf next to their source .md in docs/.)

- **Per-chapter structure (from V1 split onward):** the curriculum is also split into an independent per-chapter tree under docs/Volume <ROMAN>/Chapter <N>/ — each chapter has ...-Theory.md, ...-Labs.md (+ their .docx/.pdf) and a ...-context.md AI interaction log. **Each chapter now progresses independently** (its Theory feeds later PowerPoint/video generation). Tooling:

```
python3 scripts/split-chapters.py          # (re)create the tree;
won't overwrite edited chapters (FORCE=1 to regenerate from the
master)
```

```
scripts/export-chapters.sh
```

```
# export every chapter
```

```
Theory/Labs to docx + pdf
```

After a chapter has been edited independently, do not run `split-chapters.py` without `FORCE=1` — it would overwrite the chapter from the frozen master. Read a chapter's `...-context.md` before modifying it, and append an entry after.

13. Sources & References

A curated starting list of the tools, standards, models, and papers referenced in this plan, grouped by topic. **These should be verified and expanded as the curriculum is built** — treat as pointers, not fixed citations. Official project sites are the most stable; model/paper entries give the name + originating group so they can be located.

Programming & data science

- **Python** — python.org · **Anaconda** — anaconda.com · **Jupyter** — jupyter.org
- **NumPy** — numpy.org · **Pandas** — pandas.pydata.org · **Matplotlib** — matplotlib.org · **scikit-learn** — scikit-learn.org

Clinical data, code sets & privacy

- **ICD** (WHO) — who.int/standards/classifications · **CPT** (AMA) · **SNOMED CT** — snomed.org · **LOINC** — loinc.org
- **HIPAA** (US HHS) — hhs.gov/hipaa · **GDPR** (EU) — gdpr.eu

Interoperability standards

- **HL7** (v2.x, CDA/C-CDA) — hl7.org · **FHIR** — hl7.org/fhir · **SMART on FHIR** — smarthealthit.org
- **HAPI FHIR** (open-source FHIR server/lib) — hapifhir.io · **Synthea** (synthetic patients) — github.com/synthetichealth/synthea · **hl7apy** (Python HL7 v2) — github.com/crs4/hl7apy · **fhir.resources** / **fhirclient** (Python)

Transformers & the Hugging Face ecosystem

- **"Attention Is All You Need"** — Vaswani et al., 2017 (transformer/self-attention)
- **Hugging Face** — huggingface.co (Transformers, Datasets, model hub)

Medical LLMs (text & multimodal)

- **PubMedBERT** — Microsoft (biomedical BERT) · **Meditron** — EPFL/LLM lab (Llama-based medical LLM)
- **Clinical-Camel** — clinical LLM · **MedGemma** — Google (open medical model; text + multimodal)
- **Med-PaLM / Med-PaLM 2** — Google (Singhal et al., 2023) · **Med-PaLM M** — Google (multimodal)
- **LLaVA-Med** — Microsoft (biomedical vision-language assistant)

Fine-tuning, alignment & benchmarks

- **LoRA** — Hu et al., 2021 · **QLoRA** — Dettmers et al., 2023 · **DPO** (Direct Preference Optimization) — Rafailov et al., 2023
- Benchmarks: **MedQA** (USMLE-style), **PubMedQA**, **MedMCQA**
- **MedFineWeb** — domain-specific medical training corpus (verify source/availability)

RAG, vector databases & local inference

- **RAG** — Lewis et al., 2020 (Retrieval-Augmented Generation)
- Vector DBs: **Chroma** — trychroma.com · **Weaviate** — weaviate.io · **Pinecone** — pinecone.io · **FAISS** (Meta)
- Local serving: **Ollama** — ollama.com · **Open WebUI** — openwebui.com

Clinical datasets (research)

- **MIMIC** (de-identified ICU EHR) — PhysioNet, physionet.org · **Synthea** (synthetic, above)

Robotics & embodied AI

- **ROS 2** — ros.org · **Gazebo** — gazebo.org · **NVIDIA Isaac Sim** — developer.nvidia.com/isaac-sim
- **STAR** (Smart Tissue Autonomous Robot) — [autonomous-suturing research](https://starrobotics.org) · **RT-2 / Vision-Language-Action (VLA) models** — [robotics foundation-model research](https://openai.com/research/robotics-foundation-model)

Reference medical-AI curricula & guidance

- General medical-AI education literature and best-practice curricula (to be catalogued as the curriculum is developed).
-

End of Med-Tech Plan — V1.